```
import sqlite3
import tkinter as tk
from tkinter import messagebox, ttk
import random
import string
from datetime import datetime

# Fungsi untuk menghasilkan ID unik
def generate_id(prefix="ID", length=8):
    characters = string.ascii_uppercase + string.digits
    return prefix + ''.join(random.choices(characters, k=length))

# Kelas untuk menangani operasi database
class DatabaseHandler:
    def __init__(self, db_name="ruangan_kampus.db"):
        self.db_name = db_name
        self.init_db()

    def init_db(self):
        with sqlite3.connect(self.db_name, timeout=10) as conn:
            cursor = conn.cursor()
            # Hapus tabel Jadwal jika sudah ada (opsional)
            # cursor.execute("DROP TABLE IF EXISTS Jadwal")

            # Membuat tabel Ruangan
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS Ruangan (
                ID_Ruangan TEXT PRIMARY KEY,
                Nama_Ruangan TEXT NOT NULL,
                Kapasitas INTEGER NOT NULL
            )
            """)
```

```
            # Membuat tabel User
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS User (
                ID_User TEXT PRIMARY KEY,
                Nama TEXT NOT NULL,
                Password TEXT NOT NULL,
                Role TEXT CHECK(Role IN ('Operator', 'Mahasiswa', 'Dosen'))
NOT NULL
            )
            """)

            # Membuat tabel Jadwal
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS Jadwal (
                ID_Jadwal TEXT PRIMARY KEY,
                ID_Ruangan TEXT NOT NULL,
                Waktu_Mulai TEXT NOT NULL,
                Waktu_Selesai TEXT NOT NULL,
                Status TEXT CHECK(Status IN ('Tersedia', 'Dipakai',
'Terbooking')) NOT NULL,
                Pemesan TEXT,
                Status_Approval TEXT CHECK(Status_Approval IN ('Approved',
'Pending', 'Rejected')) DEFAULT 'Pending',
                FOREIGN KEY (ID_Ruangan) REFERENCES Ruangan(ID_Ruangan)
            )
            """)

            # Membuat tabel Peminjaman
            cursor.execute("""
            CREATE TABLE IF NOT EXISTS Peminjaman (
                ID_Peminjaman TEXT PRIMARY KEY,
                ID_Ruangan TEXT NOT NULL,
```

```
                Waktu_Mulai TEXT NOT NULL,
                Waktu_Selesai TEXT NOT NULL,
                Pemesan TEXT,
                Status TEXT CHECK(Status IN ('Menunggu', 'Disetujui',
'Ditolak')) DEFAULT 'Menunggu',
                FOREIGN KEY (ID_Ruangan) REFERENCES Ruangan(ID_Ruangan)
            )
            """)

            conn.commit()

    def tambah_user(self, id_user, nama, password, role):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("INSERT INTO User (ID_User, Nama, Password,
Role) VALUES (?, ?, ?, ?)",
                               (id_user, nama, password, role))
                conn.commit()
        except sqlite3.IntegrityError:
            # User sudah ada, tidak melakukan apa-apa
            pass
        except sqlite3.OperationalError as e:
            print(f"Error: {e}")

    def setup_default_users(self):
        default_users = [
            ("O001", "Operator1", "password", "Operator"),
            ("M001", "Mahasiswa1", "password", "Mahasiswa"),
            ("D001", "Dosen1", "password", "Dosen")
        ]
        for user in default_users:
            self.tambah_user(*user)
```

```
    def autentikasi_user(self, id_user, password):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM User WHERE ID_User = ?",
(id_user,))
                user = cursor.fetchone()

                if user and user[2] == password:
                    return user[3]  # Mengembalikan role
                else:
                    return None
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Error database: {e}")
            return None

    def get_ruangan(self):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM Ruangan")
                return cursor.fetchall()
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Error database: {e}")
            return []

    def get_daftar_ruangan(self):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("SELECT ID_Ruangan, Nama_Ruangan FROM
Ruangan")
```

```
                return cursor.fetchall()
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Error database: {e}")
            return []

    def get_jadwal(self):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("""
                    SELECT
                        j.ID_Jadwal,  -- Menambahkan ID_Jadwal
                        r.Nama_Ruangan,
                        IFNULL(j.Waktu_Mulai, 'Belum Ada Jadwal') AS
Waktu_Mulai,
                        IFNULL(j.Waktu_Selesai, 'Belum Ada Jadwal') AS
Waktu_Selesai,
                        IFNULL(j.Status, 'Tersedia') AS Status,
                        IFNULL(j.Pemesan, 'Belum Ada') AS Pemesan,
                        IFNULL(j.Status_Approval, '-') AS Status_Approval
                    FROM Ruangan r
                    LEFT JOIN Jadwal j ON r.ID_Ruangan = j.ID_Ruangan
                    ORDER BY r.ID_Ruangan, j.Waktu_Mulai
                """)
                return cursor.fetchall()
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Operasi database gagal: {e}")
            return []

    def tambah_ruangan_ke_db(self, id_ruangan, nama_ruangan, kapasitas):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
```

```
                cursor.execute("""
                    INSERT INTO Ruangan (ID_Ruangan, Nama_Ruangan, Kapasitas)
                    VALUES (?, ?, ?)
                """, (id_ruangan, nama_ruangan, kapasitas))
                conn.commit()
                return True
        except sqlite3.IntegrityError:
            messagebox.showerror("Error", "ID Ruangan sudah ada! Silakan
gunakan ID yang berbeda.")
            return False
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Operasi database gagal: {e}")
            return False


    def ajukan_peminjaman(self, id_ruangan, waktu_mulai, waktu_selesai,
pemesan):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                id_peminjaman = generate_id(prefix="P")
                cursor.execute("""
                    INSERT INTO Peminjaman (ID_Peminjaman, ID_Ruangan,
Waktu_Mulai, Waktu_Selesai, Pemesan, Status)
                    VALUES (?, ?, ?, ?, ?, ?)
                """, (id_peminjaman, id_ruangan, waktu_mulai, waktu_selesai,
pemesan, 'Menunggu'))
                conn.commit()
                return True
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Operasi database gagal: {e}")
            return False


    def approve_reject_peminjaman(self, id_peminjaman, status):
        if status not in ["Disetujui", "Ditolak"]:
```

```
            messagebox.showerror("Error", "Status tidak valid!")
            return False
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                # Jika ingin menyetujui, periksa konflik jadwal terlebih
dahulu
                if status == "Disetujui":
                    cursor.execute("""
                        SELECT ID_Ruangan, Waktu_Mulai, Waktu_Selesai
                        FROM Peminjaman
                        WHERE ID_Peminjaman = ?
                    """, (id_peminjaman,))
                    peminjaman = cursor.fetchone()
                    if peminjaman:
                        id_ruangan, waktu_mulai, waktu_selesai = peminjaman
                        if self.is_jadwal_conflict(id_ruangan, waktu_mulai,
waktu_selesai):
                            messagebox.showerror("Konflik Jadwal", "Waktu
peminjaman bertabrakan dengan jadwal yang sudah ada.")
                            return False
                # Update status di tabel Peminjaman
                cursor.execute("""
                    UPDATE Peminjaman
                    SET Status = ?
                    WHERE ID_Peminjaman = ?
                """, (status, id_peminjaman))

                if status == "Disetujui":
                    # Ambil detail peminjaman yang disetujui
                    cursor.execute("""
                        SELECT ID_Ruangan, Waktu_Mulai, Waktu_Selesai,
Pemesan
                        FROM Peminjaman
```

```
                        WHERE ID_Peminjaman = ?
                    """, (id_peminjaman,))
                    peminjaman = cursor.fetchone()

                    if peminjaman:
                        id_ruangan, waktu_mulai, waktu_selesai, pemesan =
peminjaman
                        # Buat entry baru di tabel Jadwal
                        id_jadwal = generate_id(prefix="J")
                        cursor.execute("""
                            INSERT INTO Jadwal (ID_Jadwal, ID_Ruangan,
Waktu_Mulai, Waktu_Selesai, Status, Pemesan, Status_Approval)
                            VALUES (?, ?, ?, ?, ?, ?, 'Approved')
                        """, (id_jadwal, id_ruangan, waktu_mulai,
waktu_selesai, 'Terbooking', pemesan))
                        print(f"Jadwal berhasil ditambahkan dengan ID_Jadwal:
{id_jadwal}")  # Debug
                        messagebox.showinfo("Sukses", f"Jadwal berhasil
ditambahkan dengan ID_Jadwal: {id_jadwal}")
                conn.commit()
                return True
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Operasi database gagal: {e}")
            return False

    def is_jadwal_conflict(self, id_ruangan, waktu_mulai, waktu_selesai):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("""
                    SELECT *
                    FROM Jadwal
                    WHERE ID_Ruangan = ?
                    AND Status = 'Terbooking'
```

```
                    AND (
                        (? < Waktu_Selesai AND ? > Waktu_Mulai)
                    )
                """, (id_ruangan, waktu_mulai, waktu_selesai))
                return cursor.fetchone() is not None
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Operasi database gagal: {e}")
            return True  # Asumsi terjadi konflik jika gagal mengecek

    def get_pengajuan_peminjaman(self):
        try:
            with sqlite3.connect(self.db_name, timeout=10) as conn:
                cursor = conn.cursor()
                cursor.execute("""
                    SELECT p.ID_Peminjaman, r.Nama_Ruangan, p.Waktu_Mulai,
p.Waktu_Selesai, p.Pemesan, p.Status
                    FROM Peminjaman p
                    JOIN Ruangan r ON p.ID_Ruangan = r.ID_Ruangan
                """)
                return cursor.fetchall()
        except sqlite3.OperationalError as e:
            messagebox.showerror("Error", f"Error database: {e}")
            return []

# Kelas untuk menangani autentikasi
class Auth:
    def __init__(self, db_handler):
        self.db_handler = db_handler

    def authenticate(self, id_user, password):
        return self.db_handler.autentikasi_user(id_user, password)
```

```
# Kelas UI Login
class LoginUI:
    def __init__(self, root, auth_handler, on_success):
        self.root = root
        self.auth_handler = auth_handler
        self.on_success = on_success
        self.setup_ui()

    def setup_ui(self):
        self.root.title("Login")

        tk.Label(self.root, text="ID User:").grid(row=0, column=0, padx=10,
pady=10)
        self.entry_id_user = tk.Entry(self.root)
        self.entry_id_user.grid(row=0, column=1, padx=10, pady=10)

        tk.Label(self.root, text="Password:").grid(row=1, column=0, padx=10,
pady=10)
        self.entry_password = tk.Entry(self.root, show="*")
        self.entry_password.grid(row=1, column=1, padx=10, pady=10)

        tk.Button(self.root, text="Login",
command=self.cek_login).grid(row=2, column=0, columnspan=2, pady=10)

    def cek_login(self):
        id_user = self.entry_id_user.get()
        password = self.entry_password.get()
        role = self.auth_handler.authenticate(id_user, password)

        if role:
            self.on_success(role)
            # Clear the entries
            self.entry_id_user.delete(0, tk.END)
```

```
            self.entry_password.delete(0, tk.END)
        else:
            messagebox.showerror("Login Gagal", "ID atau Password salah!")

# Kelas UI Operator
class OperatorUI:
    def __init__(self, db_handler, master):
        self.db_handler = db_handler
        self.master = master
        self.window = tk.Toplevel(master)
        self.window.title("Operator - Manajemen Ruangan dan Jadwal")
        self.setup_ui()

    def setup_ui(self):
        # Frame untuk Ruangan
        frame_ruangan = tk.LabelFrame(self.window, text="Data Ruangan")
        frame_ruangan.pack(fill="both", expand=True, padx=10, pady=5)

        columns_ruangan = ("ID Ruangan", "Nama Ruangan", "Kapasitas")
        self.tree_ruangan = ttk.Treeview(frame_ruangan,
columns=columns_ruangan, show="headings")
        for col in columns_ruangan:
            self.tree_ruangan.heading(col, text=col)
        self.tree_ruangan.pack(fill="both", expand=True)

        tk.Button(self.window, text="Tambah Ruangan",
command=self.tambah_ruangan_ui).pack(pady=5)

        # Frame untuk Jadwal
        frame_jadwal = tk.LabelFrame(self.window, text="Data Jadwal")
        frame_jadwal.pack(fill="both", expand=True, padx=10, pady=5)
```

```
        columns_jadwal = ("ID Jadwal", "Nama Ruangan", "Waktu Mulai", "Waktu
Selesai", "Status", "Pemesan", "Status Approval")

        self.tree_jadwal = ttk.Treeview(frame_jadwal, columns=columns_jadwal,
show="headings")

        for col in columns_jadwal:

            self.tree_jadwal.heading(col, text=col)

        self.tree_jadwal.pack(fill="both", expand=True)


        tk.Button(self.window, text="Tampilkan Jadwal",
command=self.tampilkan_data_jadwal).pack(pady=5)


        # Frame untuk Pengajuan Peminjaman

        frame_pengajuan = tk.LabelFrame(self.window, text="Pengajuan
Peminjaman")

        frame_pengajuan.pack(fill="both", expand=True, padx=10, pady=5)


        columns_pengajuan = ("ID Peminjaman", "Nama Ruangan", "Waktu Mulai",
"Waktu Selesai", "Pemesan", "Status")

        self.tree_pengajuan = ttk.Treeview(frame_pengajuan,
columns=columns_pengajuan, show="headings")

        for col in columns_pengajuan:

            self.tree_pengajuan.heading(col, text=col)

        self.tree_pengajuan.pack(fill="both", expand=True)


        tk.Button(self.window, text="Tampilkan Pengajuan",
command=self.tampilkan_pengajuan_peminjaman).pack(pady=5)

        tk.Button(self.window, text="Approve/Reject",
command=self.approve_reject_ui).pack(pady=5)


        # Tombol Refresh Data

        tk.Button(self.window, text="Refresh Data",
command=self.refresh_all).pack(pady=10)


        # Menampilkan data awal

        self.tampilkan_data_ruangan()

        self.tampilkan_data_jadwal()
```

```
        self.tampilkan_pengajuan_peminjaman()

    def refresh_all(self):
        self.tampilkan_data_ruangan()
        self.tampilkan_data_jadwal()
        self.tampilkan_pengajuan_peminjaman()

    def tampilkan_data_ruangan(self):
        for item in self.tree_ruangan.get_children():
            self.tree_ruangan.delete(item)
        data = self.db_handler.get_ruangan()
        for row in data:
            self.tree_ruangan.insert("", tk.END, values=row)

    def tampilkan_data_jadwal(self):
        for item in self.tree_jadwal.get_children():
            self.tree_jadwal.delete(item)
        data = self.db_handler.get_jadwal()
        for row in data:
            self.tree_jadwal.insert("", tk.END, values=row)

    def tampilkan_pengajuan_peminjaman(self):
        for item in self.tree_pengajuan.get_children():
            self.tree_pengajuan.delete(item)
        data = self.db_handler.get_pengajuan_peminjaman()
        for row in data:
            self.tree_pengajuan.insert("", tk.END, values=row)

    def approve_reject_ui(self):
        selected_item = self.tree_pengajuan.selection()
        if not selected_item:
```

```
            messagebox.showwarning("Pilih Pengajuan", "Pilih pengajuan
peminjaman yang akan diproses!")
            return


        id_peminjaman = self.tree_pengajuan.item(selected_item, "values")[0]


        def approve():
            if self.db_handler.approve_reject_peminjaman(id_peminjaman,
"Disetujui"):
                messagebox.showinfo("Sukses", "Peminjaman berhasil
disetujui!")
                self.tampilkan_pengajuan_peminjaman()
                self.tampilkan_data_jadwal()  # Refresh jadwal
                form.destroy()


        def reject():
            if self.db_handler.approve_reject_peminjaman(id_peminjaman,
"Ditolak"):
                messagebox.showinfo("Sukses", "Peminjaman berhasil ditolak!")
                self.tampilkan_pengajuan_peminjaman()
                form.destroy()


        form = tk.Toplevel(self.window)
        form.title("Konfirmasi Approve/Reject")
        tk.Label(form, text="Apakah Anda ingin menyetujui atau menolak
peminjaman ini?").pack(pady=10)
        tk.Button(form, text="Approve", command=approve).pack(pady=5)
        tk.Button(form, text="Reject", command=reject).pack(pady=5)


    def tambah_ruangan_ui(self):
        def simpan_ruangan():
            id_ruangan = entry_id_ruangan.get().strip()
            nama_ruangan = entry_nama_ruangan.get().strip()
            kapasitas = entry_kapasitas.get().strip()
```

```
            if not id_ruangan or not nama_ruangan or not kapasitas:
                messagebox.showwarning("Input Kosong", "Semua kolom harus
diisi!")
                return

            try:
                kapasitas = int(kapasitas)
                if kapasitas <= 0:
                    raise ValueError
            except ValueError:
                messagebox.showwarning("Input Salah", "Kapasitas harus berupa
angka positif!")
                return

            if self.db_handler.tambah_ruangan_ke_db(id_ruangan, nama_ruangan,
kapasitas):
                messagebox.showinfo("Sukses", "Ruangan berhasil
ditambahkan!")
                form.destroy()
                self.tampilkan_data_ruangan()

        form = tk.Toplevel(self.window)
        form.title("Tambah Ruangan")

        tk.Label(form, text="ID Ruangan:").grid(row=0, column=0, padx=10,
pady=5)
        entry_id_ruangan = tk.Entry(form)
        entry_id_ruangan.grid(row=0, column=1, padx=10, pady=5)

        tk.Label(form, text="Nama Ruangan:").grid(row=1, column=0, padx=10,
pady=5)
        entry_nama_ruangan = tk.Entry(form)
        entry_nama_ruangan.grid(row=1, column=1, padx=10, pady=5)
```

```
        tk.Label(form, text="Kapasitas:").grid(row=2, column=0, padx=10,
pady=5)
        entry_kapasitas = tk.Entry(form)
        entry_kapasitas.grid(row=2, column=1, padx=10, pady=5)

        tk.Button(form, text="Simpan Ruangan",
command=simpan_ruangan).grid(row=3, column=0, columnspan=2, pady=10)

# Kelas UI Mahasiswa dan Dosen
class MahasiswaDosenUI:
    def __init__(self, db_handler, role, master):
        self.db_handler = db_handler
        self.role = role
        self.master = master
        self.window = tk.Toplevel(master)
        self.window.title(f"Manajemen Jadwal - {self.role}")
        self.setup_ui()

    def setup_ui(self):
        # Frame untuk Daftar Ruangan
        frame_daftar_ruangan = tk.LabelFrame(self.window, text="Daftar
Ruangan")
        frame_daftar_ruangan.pack(fill="both", expand=True, padx=10, pady=5)

        columns_daftar_ruangan = ("ID Ruangan", "Nama Ruangan")
        self.tree_daftar_ruangan = ttk.Treeview(frame_daftar_ruangan,
columns=columns_daftar_ruangan, show="headings")
        for col in columns_daftar_ruangan:
            self.tree_daftar_ruangan.heading(col, text=col)
        self.tree_daftar_ruangan.pack(fill="both", expand=True)

        # Tombol Refresh Daftar Ruangan
```

```
        tk.Button(self.window, text="Refresh Daftar Ruangan",
command=self.tampilkan_daftar_ruangan).pack(pady=5)


        # Frame untuk Jadwal

        frame_jadwal = tk.LabelFrame(self.window, text="Data Jadwal")

        frame_jadwal.pack(fill="both", expand=True, padx=10, pady=5)


        columns_jadwal = ("ID Jadwal", "Nama Ruangan", "Waktu Mulai", "Waktu
Selesai", "Status", "Pemesan", "Status Approval")

        self.tree_jadwal = ttk.Treeview(frame_jadwal, columns=columns_jadwal,
show="headings")

        for col in columns_jadwal:

            self.tree_jadwal.heading(col, text=col)

        self.tree_jadwal.pack(fill="both", expand=True)


        # Tombol untuk Lihat Jadwal

        tk.Button(self.window, text="Lihat Jadwal",
command=self.tampilkan_data_jadwal).pack(pady=5)

        tk.Button(self.window, text="Ajukan Peminjaman",
command=self.ajukan_peminjaman_ui).pack(pady=5)

        tk.Button(self.window, text="Refresh Data",
command=self.tampilkan_data_jadwal).pack(pady=5)


        # Menampilkan data awal

        self.tampilkan_daftar_ruangan()

        self.tampilkan_data_jadwal()


    def tampilkan_daftar_ruangan(self):

        for item in self.tree_daftar_ruangan.get_children():

            self.tree_daftar_ruangan.delete(item)

        data = self.db_handler.get_daftar_ruangan()

        for row in data:

            self.tree_daftar_ruangan.insert("", tk.END, values=row)
```

```
    def tampilkan_data_jadwal(self):
        for item in self.tree_jadwal.get_children():
            self.tree_jadwal.delete(item)
        data = self.db_handler.get_jadwal()
        for row in data:
            self.tree_jadwal.insert("", tk.END, values=row)

    def ajukan_peminjaman_ui(self):
        def simpan_peminjaman():
            # Mengambil ID Ruangan dari Combobox
            selected_ruangan = combo_ruangan.get()
            if not selected_ruangan:
                messagebox.showwarning("Input Kosong", "Silakan pilih
ruangan!")
                return
            id_ruangan = selected_ruangan.split(" - ")[0]  # Asumsi format
"ID_Ruangan - Nama_Ruangan"

            waktu_mulai = entry_waktu_mulai.get().strip()
            waktu_selesai = entry_waktu_selesai.get().strip()
            pemesan = entry_pemesan.get().strip()

            if not waktu_mulai or not waktu_selesai or not pemesan:
                messagebox.showwarning("Input Kosong", "Semua kolom harus
diisi!")
                return

            # Validasi format waktu (misalnya, YYYY-MM-DD HH:MM)
            try:
                datetime.strptime(waktu_mulai, "%Y-%m-%d %H:%M")
                datetime.strptime(waktu_selesai, "%Y-%m-%d %H:%M")
            except ValueError:
                messagebox.showwarning("Format Salah", "Format waktu harus
YYYY-MM-DD HH:MM")
```

```
                return


            if self.db_handler.ajukan_peminjaman(id_ruangan, waktu_mulai,
waktu_selesai, pemesan):
                messagebox.showinfo("Sukses", "Peminjaman berhasil
diajukan!")
                form.destroy()
                self.tampilkan_data_jadwal()


        form = tk.Toplevel(self.window)
        form.title("Ajukan Peminjaman")


        tk.Label(form, text="Ruangan:").grid(row=0, column=0, padx=10,
pady=5)


        # Mengambil daftar ruangan untuk mengisi Combobox
        daftar_ruangan = self.db_handler.get_daftar_ruangan()
        if not daftar_ruangan:
            messagebox.showerror("Error", "Tidak ada ruangan yang tersedia!")
            form.destroy()
            return


        ruangan_options = [f"{ruangan[0]} - {ruangan[1]}" for ruangan in
daftar_ruangan]
        combo_ruangan = ttk.Combobox(form, values=ruangan_options,
state="readonly")
        combo_ruangan.grid(row=0, column=1, padx=10, pady=5)
        combo_ruangan.current(0)  # Pilih ruangan pertama secara default


        tk.Label(form, text="Waktu Mulai (YYYY-MM-DD HH:MM):").grid(row=1,
column=0, padx=10, pady=5)
        entry_waktu_mulai = tk.Entry(form)
        entry_waktu_mulai.grid(row=1, column=1, padx=10, pady=5)
```

```
        tk.Label(form, text="Waktu Selesai (YYYY-MM-DD HH:MM):").grid(row=2,
column=0, padx=10, pady=5)
        entry_waktu_selesai = tk.Entry(form)
        entry_waktu_selesai.grid(row=2, column=1, padx=10, pady=5)

        tk.Label(form, text="Pemesan:").grid(row=3, column=0, padx=10,
pady=5)
        entry_pemesan = tk.Entry(form)
        entry_pemesan.grid(row=3, column=1, padx=10, pady=5)

        tk.Button(form, text="Ajukan Peminjaman",
command=simpan_peminjaman).grid(row=4, column=0, columnspan=2, pady=10)

# Kelas untuk menjalankan aplikasi
class RoomManagementApp:
    def __init__(self):
        self.db_handler = DatabaseHandler()
        self.db_handler.setup_default_users()
        self.auth = Auth(self.db_handler)
        self.root = tk.Tk()
        self.root.geometry("300x150")
        self.login_ui = LoginUI(self.root, self.auth, self.on_login_success)
        self.root.mainloop()

    def on_login_success(self, role):
        if role == "Operator":
            OperatorUI(self.db_handler, self.root)
        elif role in ["Mahasiswa", "Dosen"]:
            MahasiswaDosenUI(self.db_handler, role, self.root)
        else:
            messagebox.showerror("Error", "Role tidak dikenali!")

# Menjalankan aplikasi
```

```
if __name__ == "__main__":
    RoomManagementApp()
```